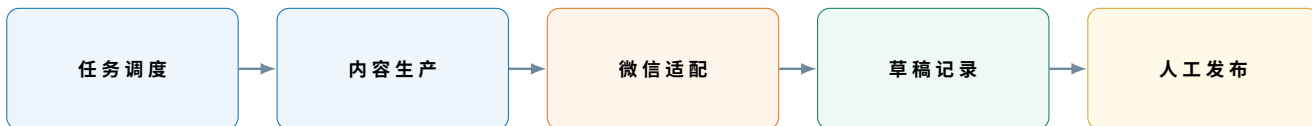


微信公众号自动推文 技术实现路线

从现有内容生产链到公众号草稿箱：架构、流程、适配与恢复

一句话架构

Scheduler 创建任务，Worker 复用现有采集与内容生产链，形成结构化文章包；微信公众号适配器完成排版、图片/封面上传和草稿创建，PostgreSQL 记录全过程状态，运营人员最终预览发布。



1 代表性公开实现给了哪些参考

使用方式

这些公开实现只用于验证“模块怎样拆、数据怎样走”的共性思路；本报告未安装、未运行、未做安全或生产验证。部分仓库未识别到许可证，公开可见不等于可复制或商用，也不建议直接拿任何一个项目上线。

代表性方案	主要解决什么	采用的技术启发	不把它当成什么
wechatpy	Python 微信接口 SDK	微信请求封装成独立客户端，业务流程不直接拼接口	不是采集、写作、审核和恢复系统
Wenyan md2wechat	Markdown 排版与草稿辅助	中间稿先渲染为内联样式 HTML，再统一处理图片与封面	不是事实治理或任务编排平台
wechat-publisher	内容到草稿的阶段式流水线	搜集、写作、配图、排版、检查、草稿分别执行	公开说明不等于生产质量已验证
auto-wx-post	token、上传、缓存与重试	凭据集中管理，媒体上传可并发，失败有限重试	不是可向模型开放的无限制发布服务

从共性中选择的五条技术原则

① 分层

内容生产、文章渲染、微信接口各自可替换。

② 中间稿

标题、摘要、正文块、图片和证据先形成稳定文章包。

③ 媒体分流

正文图片换微信 URL，封面换永久素材标识。

④ 状态持久化

任务、版本、草稿标识和错误原因写入数据库。

⑤ 人工签发

系统止于草稿；预览、修改和发布由人完成。

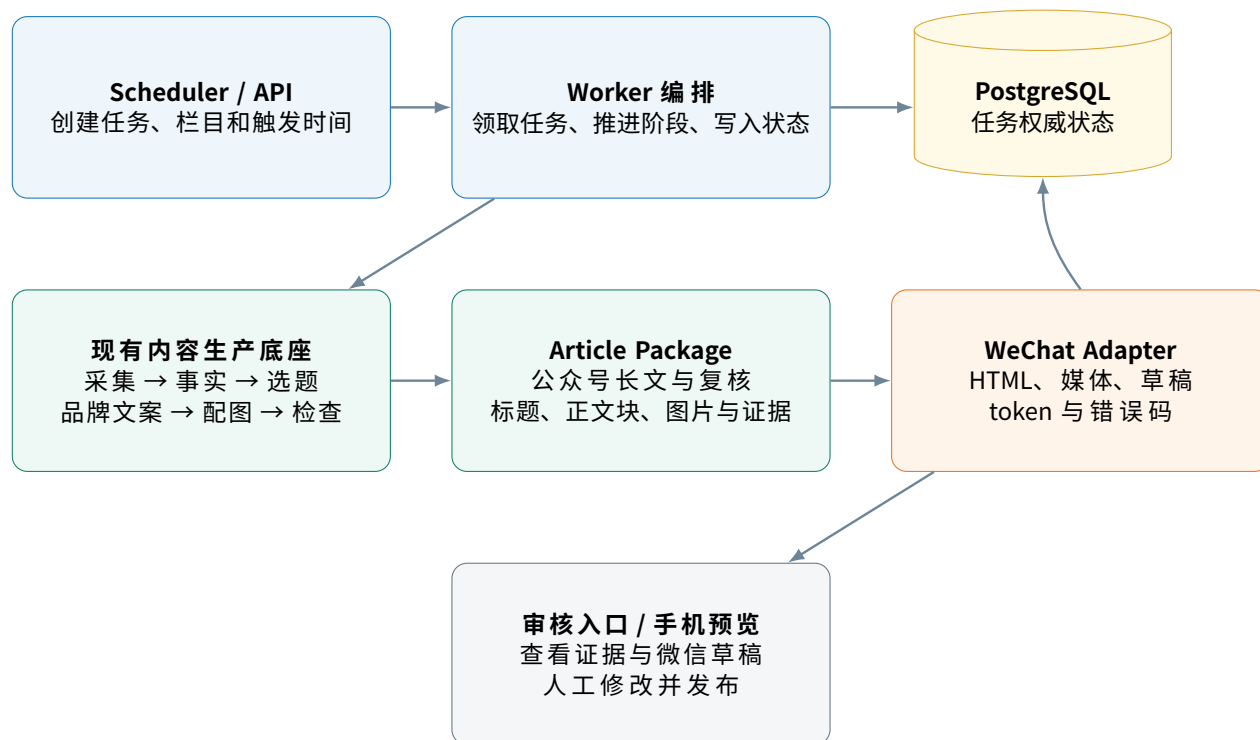
采用策略

借模块边界与可靠性思路，不复制项目整体。

明确排除

不使用浏览器模拟登录；不让写作模型接触公众号密钥；不把内容生成与微信接口写成一个不可恢复的长脚本；不把“接口返回成功”直接等同于业务完成。

2 整体架构怎样组成



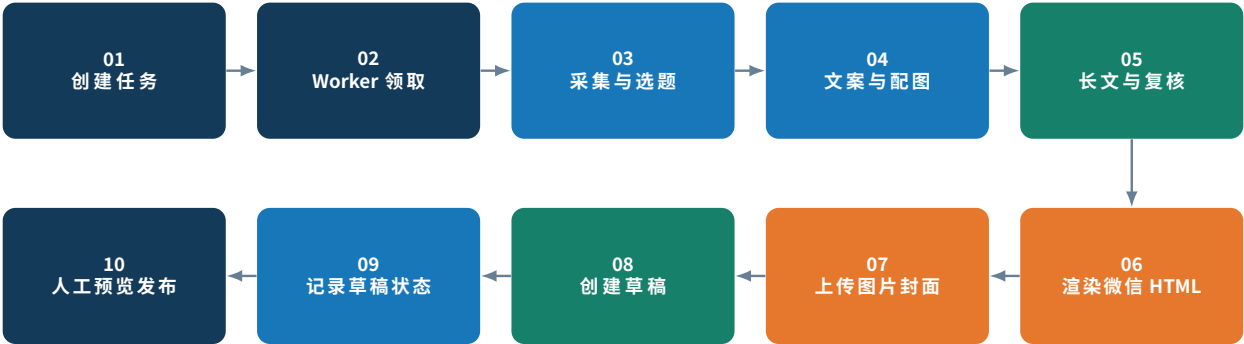
每个模块的输入和输出

模块	读入	写出
Scheduler / Worker	栏目、时间、重跑请求、待处理任务	当前阶段、租约、重试次数、完成/失败结果
现有内容生产底座	可信来源、栏目规则、品牌知识	带证据的选题、品牌文案和配图素材
Article Package	已选题与内容生产素材	按长文模板组织并复核的、可重复渲染的平台无关文章包
WeChat Adapter	已审核文章包、账号配置	微信 HTML、媒体标识、草稿标识、错误分类
PostgreSQL	每个阶段的确定结果	任务、版本、幂等键、草稿状态的权威记录

复用 / 新增边界

复用：现有可信采集、事实治理、受控选题、品牌文案、配图、检查、任务状态和失败恢复。 **新增：**Article Package 长文模板与中间稿合同、微信渲染、正文图/封面上传、草稿客户端、预览签发和草稿状态记录。

3 一篇文章怎样跑完整流程



段	执行组件	实际动作	必须落库的结果
1	Scheduler	按栏目/人工指令创建唯一任务	任务标识、业务幂等键、触发原因
2	Worker	领取任务租约，读取最近安全阶段	执行者、租约、心跳、开始时间
3-4	现有生产底座	采集、事实治理、选题、文案、配图和检查	来源、版本、检查结果和素材包位置
5-7	长文与渲染模块	按模板组织并复核文章包，生成 HTML，上传正文图与封面	文章包/复核版本、微信 URL、封面标识
8-9	草稿客户端	提交确定版本到 draft/add，记录响应	请求指纹、草稿标识、成功/错误分类
10	运营人员	手机预览、必要修改、后台发布	审核状态；发布结果后续回填

失败发生时

Worker 先把当前阶段、错误类别和已完成资源写入 PostgreSQL，再决定“有限重试”还是“等待人工处理”。恢复时从最近成功阶段继续，不重新采集、不重新生成，也不盲目重复创建草稿。

第一阶段终点

第 8-9 步成功即完成自动化验收：目标账号草稿箱能看到正确文章，系统能定位对应版本与草稿标识。第 10 步仍由授权人员完成。

4 微信公众号适配具体怎样做



子模块	实现动作	输出 / 约束
Token Manager	仅适配器读取账号凭据；集中获取并缓存 token，过期前刷新	内容模型和普通 Worker 不接触密钥；固定出口 IP 需在白名单
HTML Renderer	将 Article Package 或 Markdown 渲染为微信兼容 HTML；把 CSS 变为受控内联样式	过滤脚本和不支持标签；模板带版本，可对同一文章重复渲染
正文图片	读取已审核图片，上传“发表内容中的图片”，用返回 URL 替换正文引用	外部图片不能原样保留；每张图记录来源、哈希和微信 URL
封面素材	封面独立上传为永久素材，得到 media 标识	与正文图片分开处理；草稿使用确定的封面版本
Draft Client	组合标题、摘要、作者、HTML、封面等已审核字段，调用 draft/add	不调用正式发布接口；对请求生成指纹，避免重复草稿
Draft Record	把文章版本、请求指纹、草稿标识、时间和结果关联保存	接口成功才进入“草稿完成”；失败保留错误码和阶段

超时不能直接重试

如果 draft/add 已发出但响应丢失,先用请求指纹查本地记录,再核对平台草稿状态;无法确认时转人工处理,不直接补发,否则可能产生两份相同草稿。

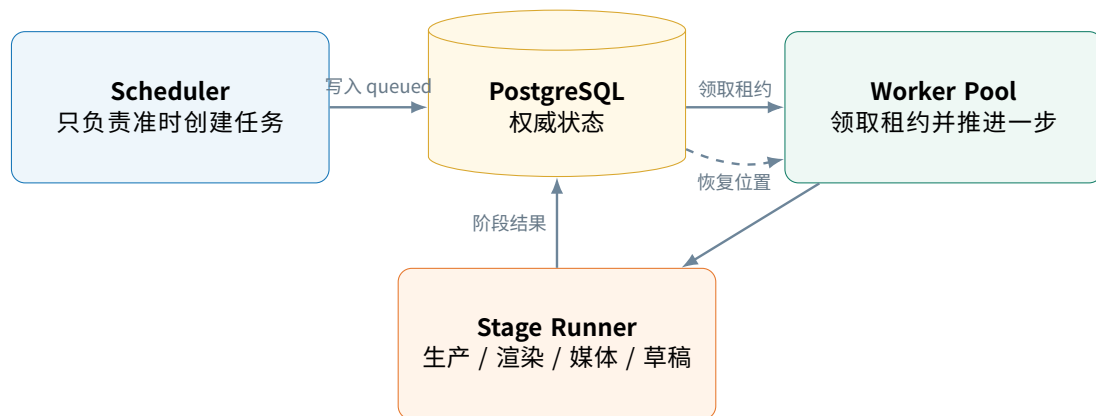
权限必须先实测

账号主体、认证状态、草稿开关、发布能力和 IP 白名单以目标公众号后台为准。权限不足时,流程明确停止并等待配置。

人工发布边界

适配器只创建草稿并返回草稿标识。运营人员在手机和公众号后台检查版式、链接、图片与全文，必要时修改后手工发布。

5 任务怎样调度，失败怎样恢复



建议的阶段进度不是一条黑盒脚本

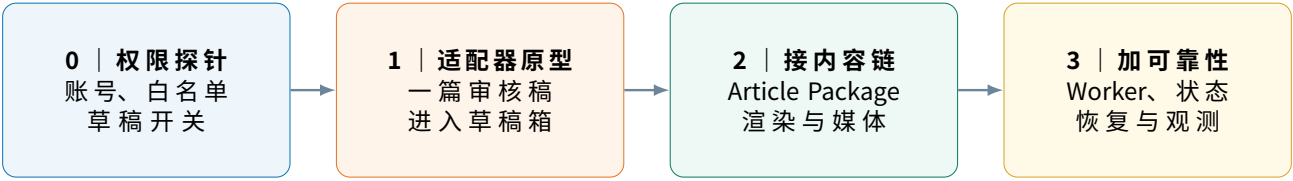


机制	怎样实现	解决的问题
PostgreSQL 权威状态	每完成一阶段，再原子地写入状态、产物位置和时间	服务重启后知道从哪里继续，不依赖内存或日志猜测
租约与心跳	Worker 领取有时限的租约，执行中定期续约	Worker 崩溃后任务可被重新领取，不会永久卡住
业务幂等键	栏目 + 内容版本 + 目标账号形成稳定唯一键	调度重复、按钮重复点击不会创建两条同一任务
请求指纹	渲染版本、图片和草稿字段形成指纹	微信超时后避免不加判断地再次创建相同草稿
有限重试	网络/限流按退避重试；权限、参数、内容错误直接停下	只重试可能恢复的问题，不制造无限循环
小型 checkpoint	编排器只保存恢复节点和必要标识，正文与状态放持久层	恢复数据可控，不把大文本塞入运行内存

三类失败的处理

可重试：短暂网络、限流、平台 5xx → 退避后有限重试。 **需修复：**权限、白名单、格式、缺图 → 停止并给出明确原因。 **结果未知：**请求已发出但响应丢失 → 先核对记录与平台状态，再决定补偿。

6 最小闭环怎样落地



技术项	复用 / 新增	最小实现	验收信号
现有内容生产底座	复用	输出带证据的选题、品牌文案和配图素材	相同输入可定位同一内容版本
Article Package	新增	按长文模板组织正文，并复核事实、品牌和版权	可脱离微信重复渲染，复核结果可追踪
微信 Renderer	新增	受控模板生成内联 HTML，过滤不支持内容	微信后台和手机预览一致
Media / Draft Adapter	新增	分别上传正文图与封面，只调用 draft/add	草稿可见，草稿标识与内容版本对应
Scheduler / Worker	复用并扩展	定时建任务、租约领取、分阶段推进	重复触发不重复建任务
PostgreSQL 状态	扩展	保存状态、产物、幂等键、请求指纹和错误类别	重启后从最近成功阶段继续
审核入口	新增	展示证据、文章版本、预览链接与草稿结果	人工确认后在后台发布

最小闭环验收

使用一篇已审核样稿完成：权限核验 → 文章包 → 微信 HTML → 正文图/封面 → draft/add → 草稿记录 → 手机预览。同一已确认任务的正常重放不重复创建草稿；结果未知时先核对平台状态，中途失败可从最近成功阶段继续。

技术结论

最稳妥的实现不是新增一个“自动发文脚本”，而是在现有内容生产链后增加可版本化文章包、可隔离微信适配器和可恢复任务状态。第一阶段做到草稿箱即完成自动化闭环，正式发布继续由人负责。

资料范围：微信官方开发文档、公开技术实现、当前项目 Agent Pipeline 规范。公开方案仅用于模块与数据流参考，未验证直接上线。实施前仍需以目标公众号后台权限和当时官方文档为准。

先把“内容 → 草稿”做成可重复、可恢复的技术链路。